

How much can self-hosted Langfuse actually handle

Langfuse publishes no throughput numbers. The self-hosting docs say "at least 2 CPUs and 4 GB of RAM per container" and "add instances once CPU exceeds 50%", and stop there.

So this was measured directly, on the Docker Compose stack in this lab.

Short answer: 395 traces per second sustained, and roughly 14 million traces per 100 GB of disk. For almost any real application, that is far more headroom than you will need, and disk will run out long before throughput does.

How it was measured

A synthetic load generator pushing traces shaped like the real agent: one trace containing about 10 observations (agent spans, two generations with token counts, `should_continue`, a tools span, and two tool spans).

No LLM calls anywhere in the loop, so this measures **Langfuse ingest**, not model latency.

Two numbers were tracked separately, because they are very different things:

1. **Client-side push**, how fast the SDK accepts and queues events
2. **End-to-end**, how long until the rows are actually queryable in ClickHouse

The second is the honest number. `langfuse.flush()` returns in 0.0 seconds because the SDK hands off to a background queue. If you only measure the flush, you will conclude Langfuse is infinitely fast, which is wrong. Server-side confirmation came from counting rows directly:

```
docker exec langfuse-clickhouse-1 clickhouse-client \  
  --query "SELECT uniqExact(trace_id), count() FROM default.events_core"
```

Test machine: **14 cores, 15 GiB RAM**, all six containers on one host, loopback network.

Throughput

6,000 traces and 60,199 events pushed in total.

Measurement	Result
Client push rate	499 traces/s
End-to-end, confirmed in ClickHouse	395 traces/s
Events per second, end-to-end	3,952 events/s
50,000 events ingested in	12.7 seconds

The worker keeps up with the client almost exactly. There was no growing backlog and no dropped events, so at this scale ingest is not the bottleneck.

Put in terms you can act on:

Your workload	Traces/second	Percent of measured capacity
10,000 traces / day	0.12	0.03%
100,000 traces / day	1.2	0.3%
1,000,000 traces / day	11.6	2.9%
34,000,000 traces / day	395	100%

A busy production agent doing 100k traces a day uses **less than half a percent** of what this single-node stack handled.

What that load costs in CPU and memory

Sampled once per second across the whole run.

Container	Idle memory	Peak memory	Peak CPU
clickhouse	557 MiB	1,000 MiB	91.5%
langfuse-web	840 MiB	937 MiB	96.5%
langfuse-worker	535 MiB	966 MiB	75.4%
minio	85 MiB	159 MiB	9.0%
postgres	48 MiB	116 MiB	4.8%
redis	9 MiB	10 MiB	3.5%
Total	2.16 GiB	~3.2 GiB	

Memory grows by about **1 GiB** from idle to full load. That is modest, and it is why the official 16 GiB recommendation is generous for anything short of sustained production ingest.

CPU is the real constraint. Three containers each hit 75 to 96 percent of a core at maximum push. Langfuse's own guidance is to add instances past 50% CPU, so **395 traces/s is the ceiling for this single node, not a comfortable operating point**. Plan for roughly half that as sustainable: call it **200 traces/s**, or 17 million a day.

Storage, which is what actually runs out

ClickHouse compresses well. Measured from `system.parts` :

Table	On disk	Uncompressed	Ratio	Rows
events_core	4.58 MiB	64.41 MiB	14.1x	60,199
events_full	4.64 MiB	64.66 MiB	13.9x	60,199

But ClickHouse is not the whole story. Large payloads go to MinIO as blobs, and that turned out to be **the bigger consumer**:

Component	Growth over 6,020 traces
ClickHouse on disk	9.67 MB
MinIO blob storage	34.12 MB
Postgres	0 MB (metadata only, does not grow with traces)
Total	43.79 MB

Which gives the number worth writing down:

7.27 MB per 1,000 traces
7.1 GB per 1,000,000 traces

Disk	Holds roughly
100 GB (the Langfuse recommendation)	14.1 million traces
500 GB	70.4 million traces
1 TB	140.8 million traces

At 1 million traces a month you would use about **7 GB a month**, so a 100 GB disk lasts over a year before you need to think about retention.

Two things to note. Postgres does not grow with trace volume, it holds users, projects and datasets, so do not size it for traffic. And this figure scales with **payload size**: this agent has small inputs and outputs, and an agent stuffing large RAG contexts into its prompts will push far more into MinIO per trace.

Caveats, because the number is not the whole truth

Being straight about what this test does not prove:

- **Ingest only.** Query performance at 100 million rows was not tested. That is usually where ClickHouse earns its keep, but it is untested here.
 - **Loopback network.** No real network latency between client and server. A remote client will be slower.
 - **Single node, no HA.** Langfuse's docs are explicit that Docker Compose "lacks high-availability, scaling capabilities, and backup functionality". Production means Kubernetes.
 - **Burst, not soak.** This ran for seconds, not days. It says nothing about ClickHouse merge pressure or disk fragmentation over months.
 - **Nothing else on the box.** All 14 cores were available.
-

Versus LangSmith

The two have completely different failure modes at volume, and that is the whole point.

	Self-hosted Langfuse	LangSmith Cloud
Throughput ceiling	395 traces/s measured , ~200/s sustainable per node	Their problem, not yours
Documented rate limit	none	6,000 requests / 10 seconds on /runs/multipart
What happens at the limit	ingest lag, then dropped events	HTTP 429, you back off
How you get more	add nodes, move to Kubernetes	pay more
Storage limit	your disk, 14M traces per 100 GB	none, but retention is 14 days by default
What actually stops you	disk and ops effort	the invoice

That last row matters more than any throughput figure. **On LangSmith you cannot run out of capacity, only out of budget. On Langfuse you cannot run out of budget, only out of disk and patience.**

What volume costs on each

Using the measured 10 observations per trace, and the published prices:

Traces / month	LangSmith (base, \$0.50/1k)	Langfuse Cloud (\$29 + \$8/100k units)	Self-hosted Langfuse
10k	\$2.50	\$29	~\$130 VM
100k	\$47.50	~\$101	~\$130 VM
1M	\$497.50	\$821	~\$130 VM + 7 GB disk
10M	\$4,997.50	\$7,949	~\$130 VM + 71 GB disk

Self-hosting is a **flat cost**. Both hosted options scale linearly with volume. The crossover sits somewhere around 130k traces a month, and past 1 million the gap becomes the entire argument.

The retention trap

LangSmith base traces are kept **14 days**. Keeping them 400 days costs **10x**, and traces with feedback attached get upgraded automatically, which is exactly what your evaluation runs produce.

Self-hosted Langfuse keeps everything until you delete it. At 7.27 MB per 1,000 traces, a year of 1M traces a month is about **85 GB**. That is one cheap disk against a 10x retention multiplier.

Sizing recommendation

Your volume	What to run
Under 100k traces / month	LangSmith or Langfuse Cloud . Self-hosting costs more than it saves.
100k to 1M / month	Either. Decide on data residency, not on price.
Over 1M / month	Self-hosted Langfuse , if you have someone to run it.
Over 17M / day	Kubernetes, multiple web and worker replicas. One node will not do it.

And regardless of volume: if the data cannot leave your network, none of the above applies. Self-host and size the disk.

Reproducing this

The load generator is `scripts/loadtest.py`. It needs a working `.env` with the `LANGFUSE_*` variables.

```
uv run python scripts/loadtest.py 5000    # 5,000 traces, ~50,000 observations
```

Then confirm server-side, which is the number that counts:

```
docker exec langfuse-clickhouse-1 clickhouse-client \
  --query "SELECT uniqExact(trace_id) AS traces, count() AS events FROM default.events_core"
```

[!WARNING] This writes real traces into your Langfuse project. This lab's instance now holds about 6,000 synthetic `obs-lab-agent` traces from the benchmark, mixed in with the genuine ones. Use a throwaway project if that matters to you.
